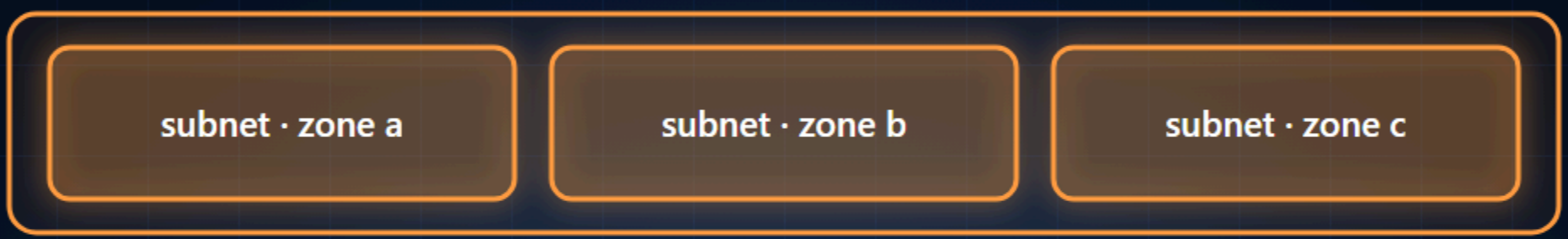
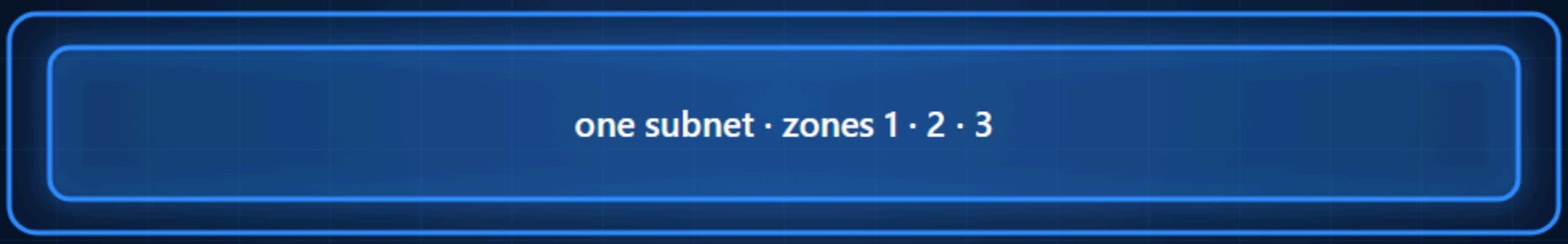
DAY 3 · AWS VS AZURE VS GCP

# Same word: "subnet." 3 different meanings.

**AWS** VPC regional · subnet = 1 zone




**Azure** VNet regional · subnet spans zones




**GCP** VPC global · subnet = 1 region




**DEJI AJISHE** · DevOps | Multi-Cloud

**Full comparison →**

WHAT & WHY

# Same concepts, different names, and different behaviors

AWS, Azure and Google Cloud all offer compute, networking, identity and storage. Most concepts map across. But a handful of **behaviors** differ in ways that change your architecture, your security model and your failure modes.

### Translates directly

VMs, images, disks, autoscaling groups, object storage, managed Kubernetes, load balancers

### Translates with traps

Resource hierarchy, IAM model, network and subnet scope, firewall attachment, VM identity

**The skill:** knowing which one-to-one mappings are safe, and which ones quietly break a design.

FOUNDATION 1 · RESOURCE HIERARCHY

# Where resources live, and how policy inherits

| | AWS | Azure | GCP |
|---|---|---|---|
| **Top** | Organization | Entra tenant → root management group | Organization |
| **Grouping** | Organizational units (OUs) | Management groups | Folders |
| **Isolation boundary** | Account | Subscription | Project |
| **Inside the boundary** | Tags | Resource groups (required) + tags | Labels |
| **Org guardrails** | SCPs | Azure Policy | Organization Policy |

→ **Billing and IAM follow the hierarchy** in all three, so design it before creating resources

→ **Every Azure resource sits in exactly one resource group**; plan their lifecycle and ownership

FOUNDATION 2 · IDENTITY

# Three IAM models, three mental models

| | AWS IAM | Azure RBAC | GCP IAM |
|---|---|---|---|
| **Permissions** | JSON policies attached to identities or resources | Role assignments at a scope (MG, subscription, RG, resource) | Allow policies binding principals to roles on a resource |
| **Workload identity** | IAM roles (assumed, temporary credentials) | Managed identities | Service accounts |
| **Inheritance** | No downward inheritance of identity policies; SCPs cap accounts | Assignments inherit down the scope | Bindings inherit down the hierarchy |
| **Explicit deny** | Yes (always wins) | Deny assignments (limited) | Deny policies |

> **Trap:** a role granted at an Azure subscription or a GCP folder applies to **everything below it**. Grant at the narrowest scope that works.

COMPUTE MAP

# Compute, containers and serverless

| | AWS | Azure | GCP |
|---|---|---|---|
| **VM** | EC2 | Virtual Machines | Compute Engine |
| **Image** | AMI | Image / Compute Gallery | Image (families) |
| **Block disk** | EBS | Managed Disks | Persistent Disk / Hyperdisk |
| **Autoscaling** | Auto Scaling group | VM Scale Sets | Managed instance groups |
| **Spare capacity** | Spot Instances | Spot VMs | Spot VMs |
| **Functions** | Lambda | Azure Functions | Cloud Run functions |
| **Containers** | ECS / Fargate | Container Apps | Cloud Run |
| **Kubernetes** | EKS | AKS | GKE |
| **Object storage** | S3 | Blob Storage | Cloud Storage |

NETWORKING · THE ROW THAT BITES

# Network and subnet scope

| AWS | Azure | GCP |
| --- | --- | --- |
| **VPC: regional**<br>Subnet = **one AZ**<br><br>HA: subnets in 2–3 AZs, instances spread across them | **VNet: regional**<br>Subnet **spans zones**<br><br>HA: one subnet, VMs pinned to zones 1/2/3 | **VPC: global**<br>Subnet = **one region**<br><br>HA: one subnet per region, VMs across zones |

→ **Porting an AWS "subnet per AZ" layout** to GCP creates needless subnets and route complexity

→ **GCP's global VPC** lets regions talk over Google's backbone with no peering

→ **Azure VNets and AWS VPCs are regional**: cross-region needs peering or a hub

NETWORKING · FIREWALLS

# Where the firewall rule actually lives

| | AWS | Azure | GCP |
|---|---|---|---|
| **Main control** | Security group | Network security group (NSG) | VPC firewall rules / policies |
| **Attached to** | Network interface (ENI) | Subnet or NIC | VPC network, targeting tags or service accounts |
| **Stateful** | Yes | Yes | Yes |
| **Deny rules** | No (allow-only) | Yes, by priority | Yes, by priority |
| **Extra layer** | Network ACLs (stateless, per subnet) | Azure Firewall | Hierarchical firewall policies |

**Design impact:** on GCP you can target rules at a **service account**, tying network access to workload identity. On AWS, "deny" needs NACLs or a firewall service.

THE PROCESS

# Translating a design from one cloud to another

01 **Map the hierarchy:** accounts → subscriptions/resource groups → projects; decide isolation boundaries

02 **Map identity:** who and what needs access, at which scope; workload identity for every service

03 **Redraw the network:** re-plan subnets for the target's scope model, not a copy of the source

04 **Translate firewall intent:** express rules as intent ("web → app on 8443"), then implement natively

05 **Map compute and scaling:** instance families, autoscaling signals, image pipeline

06 **Verify HA, quotas and limits:** zones per region, service quotas, regional availability of services

> **Rule:** translate the **intent** of a design, never the literal resources.

COMMANDS · COMPUTE WITH IDENTITY

# One VM with a workload identity, in each cloud

```
# AWS: instance profile = IAM role for the VM
aws ec2 run-instances --image-id ami-0abc1234 --instance-type t3.micro \
  --subnet-id subnet-0abc1234 \
  --iam-instance-profile Name=app-profile

# Azure: system-assigned managed identity, pinned to zone 1
az vm create -g rg-app -n vm-app --image Ubuntu2204 \
  --size Standard_B2s --zone 1 --assign-identity

# GCP: attach a dedicated service account (not the default one)
gcloud compute instances create vm-app --zone=us-central1-a \
  --machine-type=e2-small \
  --image-family=debian-12 --image-project=debian-cloud \
  --service-account=app-sa@acme-prod.iam.gserviceaccount.com \
  --scopes=cloud-platform
```

**Common thread:** no keys on the VM. Each cloud issues short-lived credentials to the workload identity.

COMMANDS · LEAST PRIVILEGE

# Grant read access to storage, the native way

```
# AWS: attach a policy to the role
aws iam attach-role-policy --role-name app-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Azure: role assignment at the narrowest scope (resource group)
az role assignment create --assignee <principal-id> \
  --role "Storage Blob Data Reader" \
  --scope /subscriptions/<sub-id>/resourceGroups/rg-app

# GCP: bind the service account to a role on the project
gcloud projects add-iam-policy-binding acme-prod \
  --member=serviceAccount:app-sa@acme-prod.iam.gserviceaccount.com \
  --role=roles/storage.objectViewer
```

→ **Managed policies and roles are a start;** scope them down to specific buckets or containers in production

INFRASTRUCTURE ORCHESTRATION

# The same intent in Terraform

| | AWS | Azure | GCP |
|---|---|---|---|
| **VM** | aws_instance | azurerm_linux_virtual_machine | google_compute_instance |
| **Network** | aws_vpc + aws_subnet | azurerm_virtual_network + azurerm_subnet | google_compute_network + google_compute_subnetwor |
| **Firewall** | aws_security_group | azurerm_network_security_group | google_compute_firewall |
| **Workload identity** | aws_iam_role + aws_iam_instance_profile | azurerm_user_assigned_identity | google_service_account |
| **Grant access** | aws_iam_role_policy_attachment | azurerm_role_assignment | google_project_iam_membe |

**Tip:** keep one module per capability per cloud behind the same inputs (see Day 2), so teams think in intent, not provider resources.

PITFALLS

# 8 mistakes when moving between clouds

- → **Copying the subnet layout** instead of re-planning for the target's scope model
- → **Granting roles too high** in Azure or GCP, where they inherit to everything below
- → **Using GCP's default compute service account** for production workloads
- → **Expecting deny rules in AWS security groups** (they're allow-only)
- → **Forgetting resource groups** when planning Azure ownership and cleanup
- → **Assuming a service exists in every region**; check regional availability first
- → **Ignoring quotas** until a deployment fails in production
- → **Mapping names, not behavior**: "equivalent" services often differ in limits and SLAs

NEED-TO-KNOWS

# Facts every multi-cloud engineer should know

- → AWS subnets live in **one AZ**; Azure subnets **span zones**; GCP subnets are **regional** in a **global** VPC
- → AWS security groups are **allow-only**; NSGs and GCP firewall rules support **deny with priority**
- → Azure and GCP role grants **inherit downward**; AWS SCPs set the **maximum** an account can do
- → In AWS, an **explicit deny** always overrides an allow
- → Every Azure resource belongs to **exactly one resource group**
- → VM identity: **instance profile** · **managed identity** · **service account**
- → GCP firewall rules can target **service accounts**, not just network tags
- → Data **egress** is usually billed in all three; plan where traffic flows

CHECKLIST

# Learning (or designing in) a second cloud

- ☐ Draw the resource hierarchy and where IAM and billing attach
- ☐ List every workload identity and its narrowest required scope
- ☐ Re-plan subnets for the target cloud's scope model
- ☐ Rewrite firewall rules as intent, then implement natively
- ☐ Confirm deny behavior and rule priority for the target
- ☐ Check regional availability for every service you need
- ☐ Check quotas and request increases before go-live
- ☐ Codify it in Terraform modules with a shared interface

HANDS-ON LAB

# Try it: One VM with a workload identity, in two clouds

01 Create a small VM with a workload identity in two clouds (no keys on the VM)

02 List subnets in each: note AZ-scoped vs zone-spanning vs regional

03 Add one firewall rule in each and note where it attaches

04 Grant read access to one storage bucket at the narrowest scope

```
# pick two clouds; commands are on the earlier pages
aws ec2 describe-subnets --query 'Subnets[].
{id:SubnetId,az:AvailabilityZone}'
az network vnet subnet list -g rg-app --vnet-name vnet-app -o table
gcloud compute networks subnets list --network=vpc-app
```

**Teardown:** Delete the VMs, firewall rules and role grants when you're done.

FURTHER READING

# Go deeper: official references

**Microsoft: Azure for AWS professionals**
learn.microsoft.com/en-us/azure/architecture/aws-professional/

**Google Cloud: Compare AWS and Azure services to Google Cloud**
cloud.google.com/docs/get-started/aws-azure-gcp-service-comparison

**AWS: Subnets for your VPC · IAM user guide**
docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html

**Microsoft: Azure RBAC overview**
learn.microsoft.com/en-us/azure/role-based-access-control/overview

**Google Cloud: VPC networks · IAM overview**
cloud.google.com/vpc/docs/vpc · cloud.google.com/iam/docs/overview

Docs change. Always check the current version before you design.

YOUR TURN

# Which cloud would you choose for a new workload, and what factors matter most?

Share your reasoning in the comments. I read and reply to every one.

**Follow Deji Ajishe**

Day 3 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a